

```
# _____ #  
# _____ #
```

```
#### Logistic Regression ####
```

```
#read in the dataset (binary response y, continuous  
explanatory x)  
orings <-read.csv("D:\\UCT\\STA2020S 2025\\orings.csv")
```

```
#####  
#slide 11  
#####
```

```
#fit the logistic regression model with the glm() function  
fit <-glm(Failure ~ Temperature, data = orings, family =  
"binomial")  
summary(fit)
```

```
#####  
#slide 16-17  
#####
```

```
#test the significance of the beta1 estimate:
```

```
#manually calculate the test statistic  
b1 <- -0.3084 #read off from summary(fit)  
se_b1 <- 0.1502 #read from summary(fit)  
zstat_b1 <- b1/se_b1 #calculates the test statistic  
zstat_b1  
#"manually" calculate the p-value  
pval_b1 <- 2*pnorm(q=zstat_b1, lower.tail = TRUE) #calculate  
the p-value for the test statistic (why do we say 2*pt() in  
this case?)  
pval_b1  
#OR just read it off from summary(fit)
```

```
#####  
#slide 18  
#####
```

```
#calculate confidence intervals for beta1 "manually"
```

```
#limits from the formulas:  
ci_b1_lower <- b1 + qnorm(p=0.025, lower.tail = T)*se_b1
```

```

ci_b1_upper <- b1 - qnorm(p=0.025, lower.tail = T)*se_b1
print(paste("Beta1 CI: (",ci_b1_lower, ci_b1_upper,")"))
#OR with R's manual function:
confint(fit) #gives CI's by interpolation, based on method
(glm) specified in "fit"
confint.default(fit)#gives confidence intervals based on
asymptotic normality (this
#is the one we calculate by hand as well)

#####
#slide 19
#####

#prediction and confidence intervals with R's built-in
functions
#predict for a single individual (with its CI and PI)
newdat <- list(Temperature = 20)
#predicted LOG-ODDS:
pred_logodds <- predict(fit, newdata = newdat, type = "link")
pred_logodds
#predicted PROBABILITIES
pred_prob <- predict(fit, newdata = newdat, type = "response")
pred_prob

#####
#slide 23
#####

#confusion matrix

#pi = 0.2
probs <- predict(fit, type = "response") #probabilities
predictions
preds <- rep(0,length=probs) #vector of 0s to store
classification in
preds[probs>0.2] = 1 #all predictions > than 0.2 is assigned
to be a 1
preds[probs<=0.2]= 0 #all predictions <= 0.2 is assigned to be
a 0
tab <- table(preds, as.numeric(orings$Failure)) #create the
confusion matrix
tab
sensit <- tab[2,2]/(tab[2,2]+tab[1,2]) #calculate sensitivity
sensit

```

```

spec <- tab[1,1]/(tab[1,1]+tab[2,1]) #calculate specificity
spec

#pi = 0.8
probs <- predict(fit, type = "response") #probabilities
predictions
preds <- rep(0,length=probs) #vector of 0s to store
classification in
preds[probs>0.8] = 1 #all predictions > than 0.8 is assigned
to be a 1
preds[probs<=0.8]= 0 #all predictions <= than 0.8 is assigned
to be a 0
tab <- table(preds, as.numeric(orings$Failure))#create the
confusion matrix
tab
sensit <- tab[2,2]/(tab[2,2]+tab[1,2]) #calculate sensitivity
sensit
spec <- tab[1,1]/(tab[1,1]+tab[2,1]) #calculate specificity
spec

```

```

# _____ #
# _____ #

```

```

#### Model Building ####

```

```

#read in the dataset
fresh <- read.csv("D:\\UCT\\STA2020S 2025\\fresh.csv")

```

```

#####
#slide 15
#####

```

```

#specify the intercept model
fit.intercept<- lm(demand ~ 1, data = fresh)
#specify the full model
fit.full <- lm(demand ~ ., data = fresh)

```

```

#create the forward selection step function
#specify direction as "forward", and specify the scope with a
lower and upper "limit"
#between which the step function should search
#start with the intercept model
step(fit.intercept, direction = "forward",

```

```
scope = list(lower = fit.intercept, upper = fit.full))
#the model with the smallest AIC is the best: full model with
all the variables
```

```
#####
```

```
#slide 17
```

```
#####
```

```
#create the backward selection step function
#specify direction as "backward", start with the full model
step(fit.full, direction = "backward")
#the model with the smallest AIC is the best: full model with
all the variables
#is the only one fit here, as removing variables did not
improve the AIC (make it smaller)
```

```
#####
```

```
#slide 19
```

```
#####
```

```
#create the backward selection step function
#specify direction as "backward", start with the full model
step(fit.intercept, direction = "both",
      scope = list(lower = fit.intercept, upper = fit.full))
#the model with the smallest AIC is the best: full model with
all the variables
#is the only one fit here, as adding/removing other variables
did not improve the
#AIC (make it smaller)
```

```
# _____ #
# _____ #
```